

Cahier des charges

Portage du jeu Snake : JavaScript → C

Ce document décrit, étape par étape, comment recréer en langage C le jeu Snake dont la version de référence (JavaScript / Canvas) est fournie en annexe. L'objectif est d'obtenir un programme console (ou graphique, en option) fonctionnellement équivalent : plateau, serpent, pomme, score, collisions, accélération progressive et redémarrage.

Auteur du projet : Anbid — IMSP, Prépa I MPSI-PCSI-Informatique

Référence : jeu Snake JavaScript (index.html + script.js)

Langage cible : C (ANSI C99 ou supérieur)

1. Analyse de la version JavaScript de référence

Avant de coder en C, il faut identifier les éléments du jeu original et leur équivalent attendu :

Élément JS	Rôle	Équivalent en C
canvasWidth / canvasHeight	Dimensions du plateau (900x600 px)	Constantes LARGEUR / HAUTEUR (en cases)
blockSize (30px)	Taille d'une case	1 case = 1 caractère ou 1 cellule de tableau
snakee (objet Snake)	Corps du serpent : tableau de [x,y]	Tableau de structures Point ou deux tableaux x[] / y[]
applee (objet Apple)	Position de la pomme	Structure Point pomme
direction ("left/right/up/down")	Direction de déplacement	Enum ou variable char (G/D/H/B)
advance()	Déplace le serpent d'une case	Fonction avancerSerpent()
checkCollision()	Détecte mur / auto-collision	Fonction collision()
isEatingApple()	Détecte si la tête touche la pomme	Fonction mangePomme()
score / speedUp()	Score et accélération (delay /= 2)	Variable score + ajustement du délai
setTimeout(refreshCanvas, delay)	Boucle de jeu animée	Boucle while avec usleep() / Sleep()
document.onkeydown	Lecture des touches fléchées	kbhit()/getch() (conio.h) ou termios (Linux)
ctx.fillRect / ctx.arc	Dessin du serpent et de la pomme	Affichage de caractères dans le terminal
gameOver() + touche Espace	Fin de partie et redémarrage	Affichage "Game Over" + relance de boucle

2. Choix techniques pour la version C

2.1 Mode d'affichage

Deux options sont possibles selon le niveau visé :

- **Mode console (recommandé pour commencer)** : le plateau est affiché avec des caractères (ex. '#' pour les murs, 'O' pour le serpent, '*' pour la pomme). On efface l'écran à chaque tour avec `system("clear")` (Linux/Mac) ou `system("cls")` (Windows).
- **Mode graphique (option avancée)** : utiliser la bibliothèque SDL2 pour reproduire fidèlement le rendu Canvas (rectangles rouges pour le serpent, cercle vert pour la pomme). À envisager seulement après une version console fonctionnelle.

2.2 Bibliothèques nécessaires

- `stdio.h` — entrées/sorties standard
- `stdlib.h` — `rand()`, `srand()` pour la position aléatoire de la pomme
- `time.h` — initialiser `srand(time(NULL))`
- `unistd.h` — `usleep()` pour temporiser la boucle (Linux/Mac)
- `termios.h` — lecture du clavier sans validation par Entrée (Linux/Mac), ou `conio.h` (`_getch`, `_kbhit`) sous Windows

2.3 Gestion du temps réel (équivalent de setTimeout)

JavaScript utilise `setTimeout(refreshCanvas, delay)` pour rejouer la boucle à intervalle régulier. En C console, il n'y a pas d'équivalent direct : on simule cela avec une boucle infinie `while(1)` qui contient un `usleep(delay * 1000)` (delay en millisecondes) et qui vérifie à chaque tour si une touche a été pressée (sans bloquer le programme).

3. Étapes de développement (ordre conseillé)

Étape 1 — Mise en place du projet

- Créer un fichier `snake.c` et un `Makefile` simple (ou compiler directement avec `gcc snake.c -o snake`).
- Définir les constantes globales : `LARGEUR`, `HAUTEUR` (équivalent de `widthInBlocks` / `heightInBlocks`), `TAILLE_MAX_SERPENT`.

Étape 2 — Structures de données

- Définir une structure `Point` avec deux champs entiers `x` et `y` (équivalent du tableau `[x, y]` en JS).
- Représenter le serpent par un tableau de `Point` (taille fixe, ex. 100) + un entier `longueur` (équivalent de `body` en JS, qui est une liste dynamique).
- Représenter la pomme par une variable `Point pomme`.
- Définir une variable `direction` (un char ou un enum `{HAUT, BAS, GAUCHE, DROITE}`) équivalent de la chaîne `"left"/"right"/"up"/"down"`.
- Définir `int score = 0`; et `int delay = 100`; (en millisecondes, comme dans le JS).

Étape 3 — Initialisation (équivalent de `init()`)

- Initialiser le serpent avec 5 cases alignées horizontalement, exactement comme `[[6,4], [5,4], [4,4], [3,4], [2,4]]` et direction initiale = droite.
- Placer la pomme avec une position aléatoire initiale, par exemple (10,10).
- Initialiser `srand(time(NULL))` pour que les positions aléatoires changent à chaque exécution.

Étape 4 — Affichage du plateau (équivalent de `draw()`)

- Écrire une fonction `afficherPlateau()` qui : efface l'écran (`system("clear")`), affiche le score en haut, dessine une bordure de '#', puis parcourt chaque case du plateau : si elle correspond à la tête du serpent afficher 'O', si elle correspond au corps afficher 'o', si elle correspond à la pomme afficher '*', sinon afficher un espace.
- Cela remplace `ctx.fillRect` (serpent rouge) et `ctx.arc` (pomme verte).

Étape 5 — Déplacement du serpent (équivalent de `advance()`)

- Écrire `avancerSerpent()` : calculer la nouvelle position de la tête selon `direction` (`x-1`, `x+1`, `y-1` ou `y+1`, comme dans le switch JS).
- Décaler tous les segments du corps vers la position du précédent (équivalent de `unshift + pop` en JS) : on insère la nouvelle tête en position 0 et on supprime la dernière case, SAUF si la pomme vient d'être mangée (alors on ne supprime pas la queue, le serpent grandit).

Étape 6 — Gestion de la direction (équivalent de `setDirection()`)

- Écrire `changerDirection(nouvelle_direction)` qui empêche un demi-tour direct : si la direction actuelle est horizontale (gauche/droite), seules les directions verticales (haut/bas) sont acceptées, et inversement — reproduire exactement la logique des `allowedDirections` du JS.

Étape 7 — Lecture du clavier en temps réel (équivalent de `document.onkeydown`)

- Sous Linux/Mac : configurer le terminal en mode "non-canonique" avec `termios` (désactiver l'attente de la touche Entrée) et utiliser une fonction non-bloquante pour vérifier si une touche est disponible.
- Sous Windows : utiliser `_kbhit()` et `_getch()` de `conio.h`.
- Associer les flèches directionnelles (ou à défaut Z/Q/S/D) aux directions haut/gauche/bas/droite, et la touche Espace au redémarrage (comme `keyCode 32` dans le JS).

Étape 8 — Détection des collisions (équivalent de `checkCollision()`)

- Écrire `collisionMur()` : vérifier si la tête sort des bornes `[0, LARGEUR-1] x [0, HAUTEUR-1]`.
- Écrire `collisionCorps()` : parcourir tous les segments du corps sauf la tête et vérifier si l'un d'eux a les mêmes coordonnées que la tête.
- Si l'une des deux fonctions renvoie vrai, appeler `finDePartie()`.

Étape 9 — Gestion de la pomme (équivalent de `isEatingApple()` et `isOnSnake()`)

- Écrire `mangePomme()` : comparer la position de la tête à celle de la pomme.
- Si la pomme est mangée : incrémenter `score`, ne pas raccourcir le serpent à ce tour, et tirer une nouvelle position aléatoire pour la pomme.
- Écrire `pommeSurSerpent()` qui vérifie que la nouvelle position tirée n'est pas déjà occupée par le serpent (reproduire la boucle `do { ... } while (applee.isOnSnake(snakee))` du JS).

Étape 10 — Accélération (équivalent de `speedUp()`)

- Toutes les 5 pommes mangées (`score % 5 == 0`), diviser `delay` par 2, exactement comme dans le JS.
- Attention : fixer une valeur plancher (ex. ne jamais descendre sous 10 ms) pour éviter une boucle trop rapide.

Étape 11 — Fin de partie et redémarrage (équivalent de `gameOver()` et `restart()`)

- Écrire `finDePartie()` : afficher "GAME OVER" et le score final, ainsi que le message "Appuyez sur Espace pour rejouer" (texte identique au message du JS).
- Attendre que l'utilisateur appuie sur Espace : si oui, réinitialiser toutes les variables (serpent, pomme, score, `delay = 100`) comme dans `restart()` et relancer la boucle de jeu ; si une autre touche (ex. 'q') est pressée, quitter le programme.

Étape 12 — Boucle principale (équivalent de `refreshCanvas()`)

- Dans `main()`, écrire la boucle de jeu : tant que le jeu n'est pas terminé : 1) lire une éventuelle touche et mettre à jour la direction, 2) avancer le serpent, 3) tester les collisions -> fin de partie si besoin, 4) tester si la pomme est mangée -> mise à jour score/pomme/vitesse, 5) afficher le plateau, 6) attendre `delay` millisecondes (`usleep(delay*1000)`).

Étape 13 — Tests et finitions

- Tester chaque cas : collision avec un mur, collision avec soi-même, manger une pomme (croissance + score), accélération après 5 pommes, redémarrage avec Espace.

- Restaurer les paramètres du terminal à la sortie du programme (remettre `termios` en mode normal) pour éviter de "casser" le terminal après exécution.
- Ajouter des commentaires en français expliquant chaque fonction, pour faciliter la relecture.

4. Tableau récapitulatif des correspondances fonctions JS -> C

Fonction JavaScript	Fonction C proposée
init()	initialiserJeu()
refreshCanvas()	boucle principale dans main()
Snake.advance()	avancerSerpent()
Snake.setDirection()	changerDirection()
Snake.checkCollision()	collisionMur() + collisionCorps()
Snake.isEatingApple()	mangePomme()
Snake.draw()	afficherPlateau() (partie serpent)
Apple.setNewPosition()	nouvellePositionPomme()
Apple.isOnSnake()	pommeSurSerpent()
Apple.draw()	afficherPlateau() (partie pomme)
speedUp()	accelerer() / mise a jour de delay
gameOver()	finDePartie()
restart()	reinitialiserJeu()
document.onkeydown	lectureClavier()

5. Squelette de départ (structures et prototypes)

Voici un point de départ pour le fichier `snake.c` (structures, constantes et prototypes seulement — à compléter en suivant les étapes ci-dessus) :

```
#define LARGEUR 30
#define HAUTEUR 20
#define TAILLE_MAX 100

typedef struct {
    int x;
    int y;
} Point;

typedef enum { HAUT, BAS, GAUCHE, DROITE } Direction;

Point serpent[TAILLE_MAX];
int longueurSerpent;
Direction direction;
Point pomme;
int score;
int delay; // en millisecondes
int enCours; // 1 = partie en cours, 0 = game over

void initialiserJeu(void);
void afficherPlateau(void);
void avancerSerpent(void);
void changerDirection(Direction nouvelle);
int collisionMur(void);
```

```
int collisionCorps(void);
int mangePomme(void);
int pommeSurSerpent(void);
void nouvellePositionPomme(void);
void accelerer(void);
void finDePartie(void);
void reinitialiserJeu(void);
void lectureClavier(void);
```

Remarque : pour la lecture clavier non bloquante sous Linux, il faudra configurer le terminal avec `tcgetattr / tcsetattr` (désactivation du mode canonique et de l'écho) et utiliser `fcntl` pour passer l'entrée standard en mode non bloquant.

6. Critères de validation finale

- Le serpent se déplace dans les 4 directions et ne peut pas faire de demi-tour instantané.
- La partie se termine en cas de sortie du plateau ou de collision avec son propre corps.
- Manger une pomme augmente le score, fait grandir le serpent et fait apparaître une nouvelle pomme à une position libre.
- Tous les 5 points, le jeu accélère (delay divisé par 2, avec une limite minimale).
- À la fin de partie, un message "GAME OVER" et le score s'affichent, et la touche Espace relance une nouvelle partie.
- Le terminal retrouve son état normal à la fermeture du programme.